

MongoDB Atlas vs Aurora PostgreSQL

A greenfield comparison across Features, Cost, Security, and Latency

This document answers a different question than the earlier decision brief: not "should we migrate the existing system," but "if EduProva were being architected today, with no existing integration, which database would we choose?" The comparison is structured around the four pillars that matter most for this kind of decision.

Verdict: If designed from scratch, Aurora PostgreSQL is the stronger default for EduProva's core data (users, enrollments, payments, subscriptions), with JSONB covering the flexible-content use cases MongoDB is usually chosen for.

1. Why the Answer Changes When Starting Fresh

In the current EduProva system, MongoDB Atlas wins mainly because the application is already built around it — migration risk, not technical fit, was the deciding factor. Remove that constraint, and the decision comes down to what EduProva's data actually looks like:

- **Relational core:** users, enrollments, courses, instructors, payments, subscriptions, orders — these have real foreign-key relationships and need atomic, all-or-nothing transactions, especially anywhere money is involved.
- **Flexible edges:** course content blocks, freelance listing metadata, profile fields — these benefit from a flexible shape.
- A from-scratch design doesn't have to choose one model exclusively — PostgreSQL's JSONB column type stores flexible/document data inside a relational database, covering most of Mongo's original advantage.

That's the central reason the recommendation shifts: Aurora PostgreSQL can absorb MongoDB's main strength (schema flexibility) via JSONB, while MongoDB cannot natively absorb Aurora's main strength (relational integrity, multi-table transactions).

2. Scorecard

Pillar	MongoDB Atlas	Aurora PostgreSQL	Winner
Features	Flexible schema, horizontal sharding, Atlas Search, Change Streams	Relational + JSONB hybrid, ACID transactions, native joins, extensions (pg_vector, PostGIS)	Aurora
Cost	Bundled pricing; minimum 3-node replica set from day one	Serverless v2 scales near-zero for early-stage/unpredictable traffic	Aurora
Security	Strong (TLS, RBAC, encryption); Atlas is a separate vendor boundary outside the AWS account	IAM-native auth, same-VPC Security Groups, single AWS account boundary	Aurora
Latency	Cross-vendor hop unless PrivateLink/Peering is configured	Same VPC as EKS by default — no external network hop	Aurora

Aurora doesn't win because MongoDB is a weak product — it wins because EduProva's specific mix of relational, money-bearing data and a fully-AWS-native stack plays to Aurora's strengths more than to MongoDB's.

3. Features

Aurora PostgreSQL (from scratch)

- Full relational model — enrollments → courses → instructors, users → subscriptions → payments, enforced by foreign keys at the database level, not just app code.
- ACID transactions across multiple tables — e.g. deduct balance + insert payment record + update enrollment status, all succeed or all roll back together.
- JSONB columns give MongoDB-style flexible fields exactly where needed (course content blocks, listing metadata) without giving up relational integrity elsewhere.
- Native SQL joins and aggregations make reporting/analytics (revenue by course, enrollment trends) far simpler than MongoDB's aggregation pipeline.
- Up to 15 read replicas, Aurora Global Database, Aurora Serverless v2 for automatic capacity scaling.
- Extensions available: pg_vector (AI-based course recommendations), PostGIS, full-text search.

MongoDB Atlas (from scratch)

- Best when nearly all data is naturally document-shaped and relationships between entities are shallow.
- Horizontal sharding is simpler to reason about than relational sharding, if the app is expected to scale to very large, simple-access-pattern datasets.
- Atlas Search and Change Streams remain genuinely useful for listings search and real-time notifications, but Aurora + a search extension (or a lightweight search layer) can cover the same need.

Winner: Aurora PostgreSQL — because EduProva's core entities need relational integrity that MongoDB doesn't natively provide, while Aurora's JSONB largely closes the flexibility gap.

4. Cost

Cost	MongoDB Atlas	Aurora PostgreSQL
Minimum production footprint	3-node replica set required from day one (no single-node production tier)	Can start on a single Aurora Serverless v2 instance and scale up
Early-stage / unpredictable traffic	Bundled per-node pricing regardless of actual usage	Serverless v2 (ACUs) scales down near-zero when idle — pay for what's used
At scale	Predictable bundled price (compute + storage + HA + backup)	Itemized (compute + storage + IOPS) — needs monitoring, but can be cheaper with reserved capacity
Operational cost	Fully managed, no self-hosting	Fully managed, no self-hosting

Winner: Aurora PostgreSQL — for a new product with unknown/early traffic, Serverless v2's scale-to-near-zero avoids paying for a fixed 3-node baseline before there's real usage to justify it.

5. Security

	MongoDB Atlas	Aurora PostgreSQL
Encryption at rest	AES-256 by default	AWS KMS-backed
Encryption in transit	TLS required	SSL/TLS
Network boundary	Separate vendor account/VPC — PrivateLink or IP whitelist needed to reach it	Lives inside the same AWS account/VPC as EKS from the start
Authentication	SCRAM / X.509 certificates	IAM database authentication — short-lived 15-minute tokens, no long-lived DB password to rotate or leak
Authorization	Custom RBAC roles	Native PostgreSQL roles + IAM policy integration
Audit trail	Atlas-side access logs	AWS CloudTrail + native PostgreSQL logging, unified with the rest of the AWS audit trail

Winner: Aurora PostgreSQL — not because Atlas is insecure (it isn't), but because keeping the database inside the same AWS account/VPC as EKS removes an entire external trust boundary, and IAM auth means one fewer long-lived secret to manage across the stack.

6. Latency

- **MongoDB Atlas:** even in the same AWS region, Atlas runs in MongoDB's own AWS account. Reaching it means either a public endpoint (IP whitelist) or setting up VPC Peering / PrivateLink to get private, low-latency access — an extra piece of infrastructure to build and maintain.
- **Aurora PostgreSQL:** in a from-scratch AWS-native build, Aurora sits directly inside the same VPC as the EKS cluster. Backend pods reach it over a Security Group rule — no cross-account networking to set up at all.
- **Both are equally fast once properly configured in the same region** — the difference is that Aurora's low latency is the default, while Atlas's low latency is something you have to deliberately configure (PrivateLink) rather than get automatically.

Winner: Aurora PostgreSQL — by default, for a greenfield fully-AWS build. Atlas can match it, but only after additional network setup.

7. Final Verdict

For a from-scratch EduProva build: choose Aurora PostgreSQL as the primary database. Use JSONB columns for flexible content (course modules, listing metadata) instead of a separate document database. Reserve MongoDB Atlas for a scenario where the product is overwhelmingly document-shaped with very shallow relationships between entities — which is not the case here.

Important nuance

This is an engineering judgment based on EduProva's described data (payments, subscriptions, enrollments, freelance contracts) and its fully-AWS stack — not a universal claim that Aurora beats MongoDB in general. Many successful platforms run well on MongoDB. The deciding factors here are: (1) the presence of money-bearing, multi-table transactional data, and (2) an all-AWS infrastructure where Aurora's same-VPC placement is a natural fit.

How to explain this in a presentation / interview

"If we were designing EduProva from scratch, I'd choose Aurora PostgreSQL over MongoDB Atlas. Our core data — enrollments, payments, subscriptions — is relational and needs atomic transactions, which Aurora gives natively. PostgreSQL's JSONB columns cover the flexible-content cases that would normally push us toward MongoDB, so we don't lose that flexibility. And since our whole stack runs on AWS, Aurora sits in the same VPC as our EKS workloads by default — better latency and a simpler security boundary than a separate managed database vendor."

Greenfield comparison prepared for EduProva project presentation.